Unified Engineering Workbench

COMMON INTERNSHIP ASSIGNMENT | 15 CALENDAR DAYS | 100 POINTS

Build a small, reliable engineering workbench that imports geometry, solves a thermal problem, visualizes results, runs parallel workloads, and represents embedded hardware. The same requirements and rubric apply to every applicant, regardless of preferred internship area.

1. Scope, time and permitted tools

Submit individually within 15 calendar days of receiving this brief. Target effort: 40-50 hours; record actual effort. Prioritize a working, tested prototype over production-scale features. A web interface, desktop application or packaged executable is acceptable. Any frontend language is permitted; all application backend logic, numerical computation, file processing and concurrency must be C++17 or later. Build/packaging scripts are permitted, but must not implement backend features.

You may use any GPT/AI assistant and openly available libraries. Disclose AI assistance, reused code, dependencies and licences. No paid AI plan, commercial solver licence, physical board or specialist CAD software is required. Use synthetic or openly licensed data only. This is an assessment prototype, not production work.

2. Task A: Interface, backend and one-click execution

Provide one coherent interface with Geometry, Thermal Analysis, Performance and Hardware views. Include file selection, parameter validation, progress/status, actionable error messages, and result export. Keep the UI responsive during computation; handle cancellation or safe completion without corrupting results. Separate the C++ core from presentation and transport code.

Target Windows 11 x64. After ZIP extraction, double-clicking Launch.exe must open the working application without terminal commands, PowerShell, administrator rights, package installation or a first-run build. Bundle redistributable runtime dependencies lawfully. Include a VS Code or Antigravity run configuration that starts the same packaged application in one action after the documented configuration is selected. Document any editor extension prerequisite; the standalone launcher must not require the editor or internet. Provide CMake source-build instructions separately.

3. Task B: Existing CAD / ANSYS data visualization

Support (i) an existing ASCII STL geometry file and (ii) a CSV export of ANSYS nodal temperature results with columns node_id,x_m,y_m,z_m,temperature_C. Supply one small, openly licensed or self-created example of each, its provenance, units and export/schema notes. A self-created CSV must be labelled synthetic, not represented as an actual ANSYS export. Native ANSYS project/result formats and native CAD solids are outside scope.

Display STL triangles with rotate, pan, zoom and fit-to-view. Display the CSV nodes as a coloured 3D point cloud with a temperature legend, units and node inspection. Mesh-to-result registration is not required. Parse and validate both formats in C++; frontend rendering libraries are allowed. Reject malformed/non-finite data, explain empty files, and document tested file-size limits.

4. Task C: Mechanical thermal FEA and interdisciplinary adoption

Implement a steady-state, one-dimensional thermal finite-element solver for a uniform rod using two-node linear elements. Use L = 1 m, A = 1e-4 m2, k = 10 W/(m K), uniform volumetric heat generation q = 10,000 W/m3, T(0) = 100 C and T(L) = 20 C. Assume constant properties and insulated lateral surfaces. Solve -d/dx(kA dT/dx) = qA; assemble element stiffness and load contributions and impose both prescribed temperatures correctly.

For N = 10, 20, 40 and 80 uniform elements, plot temperature against x and compare with T_exact(x) = 100 - 80x/L + [q/(2k)]x(L-x). Report maximum absolute error at element midpoints using FE interpolation, not only at nodes, and show mesh convergence. Plot units and export the underlying CSV. Explain assumptions, units, boundary conditions and one debugging insight. We explicitly assess interdisciplinary learning and adoption, not prior mechanical-engineering coursework.

Implementation evidence & submission

5. Task D: Multicore / multithreaded execution

Implement serial and CPU-parallel versions of a batch of independent thermal solves in C++. Use a bounded worker pool, standard threads or OpenMP; do not merely start background UI work. Provide a reproducible batch generator with a fixed seed, varied positive k and q, and configurable case count and mesh size. Use identical cases and numerical methods in both versions. Keep each solve independent and results in deterministic input order.

Measure wall-clock compute time for 1, 2 and up to 4 workers, capped at available logical CPUs. Report CPU, physical/logical core counts, RAM, compiler, optimisation flags, workload, warm-up, five measured repetitions, median time, speedup T1/Tp and efficiency (T1/Tp)/p. Exclude file loading, rendering and compilation from compute timings; report launch-to-ready time separately. Verify serial/parallel agreement with max absolute temperature difference <= 1e-8 C. Explain overhead and limitations; no minimum speedup or high-end hardware is required. On a single-logical-CPU machine, demonstrate the parallel path and disclose that speedup cannot be assessed.

6. Task E: Embedded-system integration design

Design a modular host-connected workbench for one microcontroller board, one FPGA board and one JTAG debug adapter. Provide a labelled block diagram and a concise interface/pin-voltage table covering power, grounding, voltage domains, level shifting, connectors, host communication, JTAG signal ownership and device-specific pinout assumptions. Explain why a JTAG adapter is a debug interface, not a generic interchangeable compute board; do not assume all boards share a connector, voltage or protocol.

Implement C++ mock adapters for device discovery, capability reporting, connect/disconnect and one telemetry message. Demonstrate all three device types in the UI and one recoverable failure, such as a timeout. Clearly label simulated behaviour. Include a safe integration plan and limitations. Physical wiring, PCB fabrication, firmware deployment and real FPGA programming are not required.

7. Task F: Simple handwritten backpropagation

For one regression sample x = 1, target y = 2, use h = tanh(wx + b), prediction y_hat = vh + c, and loss J = 0.5(y_hat - y)^2. Start with w = 0.5, b = 0, v = 1, c = 0 and learning rate 0.1. Handwrite the forward pass, chain-rule derivatives for all four parameters, and one simultaneous gradient-descent update using gradients from the original parameters. Include numerical values and a short explanation of gradient direction. Submit legible scans/photos, maximum two pages.

Implement the same calculation in C++ without automatic differentiation. Check each initial gradient by central finite differences with epsilon = 1e-6 and absolute error <= 1e-5. Plot loss over 100 subsequent training steps, using the same learning rate, and explain the result. AI may help you learn, but you must understand and defend the derivation.

8. Required ZIP contents and evidence

Name the archive CandidateID_Workbench.zip. Include: Launch.exe and packaged application; src/ with frontend and C++ backend; CMakeLists.txt and dependency lock/version information; editor run configuration; data/ with provenance; tests/; results/ with raw CSV and labelled plots; docs/; README; and version-control evidence. Use relative paths, no credentials, no personal data and no machine-specific absolute paths.

The README must state prerequisites, one-click steps, architecture, supported formats, test/build steps and known limitations. Include a report of at most six pages, excluding the two-page handwritten appendix: design choices, FEA verification, performance results, embedded design, test outcomes and limitations. Include a 3-5 minute demonstration video, or an equivalent timestamped screenshot walkthrough with logs.

Provide a Git bundle or local repository with meaningful development history, plus a readable commit log. Commit count alone earns no credit. Include an AI-use log with tool/model names where available, representative prompts and relevant outputs, your edits, validation evidence, and at least one incorrect or unsuitable suggestion you identified. If none occurred, document a deliberate challenge test and its outcome. Do not fabricate interactions. Include a short ownership statement identifying reused and original work.

Published evaluation & feedback

All candidates are assessed on the same 100-point rubric. Web and desktop submissions are equivalent: interface quality means usability and clarity, not a preference for a particular framework. We evaluate verified engineering decisions and understanding, not visual polish alone or the amount of AI output.

Criterion	Max.	Evidence assessed
Interface / webpage design	8	Clear workflow, readable layout, responsive status, validation and usable plots.
C++ backend and testing	12	Working integration, separation of concerns, robust error handling; positive, negative and numerical tests.
Folder architecture	6	Clean module boundaries, sensible folders, relative paths and controlled dependencies.
Code readability	6	Clear naming, focused functions, consistent style, useful comments and maintainable interfaces.
Version control	5	Traceable, meaningful changes; sensible exclusions and dependency/version records.
Thermal FEA and adoption	14	Correct formulation/assembly/BCs (6), analytical verification and convergence (5), assumptions and learning explanation (3).
CAD / ANSYS visualization	10	Both required imports, correct units/legend, interaction, provenance and malformed-file handling.
Multicore performance	10	Real parallel work (4), reproducible timing and agreement checks (4), honest analysis (2).
Embedded integration	8	Coherent modular design and electrical/interface assumptions (4); working mocks and recovery (4).
Backpropagation	7	Correct handwritten chain rule and update (4); C++ gradient check and loss plot (3).
Prompt engineering / AI judgement	8	Clear prompts, iterative refinement, independent verification, disclosure and critical judgement.
One-click delivery / report	6	Clean-machine launch and editor configuration (3); reproducible evidence, report and limitations (3).

How points are awarded

For each criterion or stated subcriterion, use these anchors: 0% = absent/unverifiable; 25% = attempted with major faults; 50% = partly working with material gaps; 75% = mostly correct with minor gaps; 100% = complete, correct and supported by reproducible evidence. Interpolate only with a written reason; round criterion scores to 0.5 points. No double penalty for one defect unless it independently affects two criteria. No bonus for expensive hardware, paid AI tools or unsupported feature count.

Shortlisting rules and candidate feedback

80-100: strong interview shortlist; 50-80: interview shortlist; below 50: not shortlisted. Both shortlist bands also require at least 6/12 in C++ backend, 7/14 in thermal FEA and 3/6 in delivery/report, a working C++ backend, and a successful standalone launch. A packaging-only failure receives one clarification/repackaging opportunity within 48 hours, with no new feature work. If unresolved, the launch gate fails; still return the scored rubric.

A brief verification discussion may ask you to explain a chosen function, reproduce a test and make one small change. Any resulting score revision must cite evidence against the same rubric. Disclosed AI use is not misconduct; suspected copying or fabricated evidence must be discussed with the candidate before an integrity decision. No automated AI-detection score is used.

Every candidate receives criterion-level scores, total /100, gate outcomes, two evidence-based strengths, two improvement priorities and the specific shortlist/non-shortlist reason. Shortlisting means progression to interview, not an internship offer. Equal scores receive equal assessment outcomes; any later capacity-based decision is communicated separately.